

Two-Phase Locking Protocol (2PL)

GATE & University Exam High-Yield Revision Notes

CONCURRENCY CONTROL

DATABASE SYSTEMS

SERIALIZABILITY

1. Overview & Evolutionary Flow

Two-Phase Locking (2PL) is an enhancement of the basic Shared-Exclusive Locking Protocol. It enforces systematic rules governing lock acquisitions and releases to guarantee conflict serializability in concurrent executions.



2. Mechanics of the Two Phases

1. Growing Phase

- **Lock Acquisition:** Allowed ($S(X)$, $X(X)$).
- **Unlocking:** Strictly PROHIBITED.
- Transaction continually accumulates resources.

2. Shrinking Phase

- **Unlocking:** Allowed ($U(X)$).
- **Lock Acquisition:** Strictly PROHIBITED.
- Begins immediately at the first unlock operation.

 **Golden Rule of 2PL:** Once a transaction releases its very first lock, it transitions permanently into the Shrinking Phase. It can *never* acquire another lock.

3. Lock Point & Serial Order

The **Lock Point** is the exact instance where a transaction acquires its *final lock* (just before the first unlock begins the shrinking phase).



Earlier Lock Point Timestamp $\Rightarrow$ Precedence in Serializability Order

If $\text{Lock_Point}(T1) < \text{Lock_Point}(T2)$, then the equivalent serial schedule is $T1 \rightarrow T2$.

4. How 2PL Guarantees Serializability

By preventing premature lock releases and enforcing lock conflicts, 2PL forces dependent transactions to wait until the lock holder completes its lock acquisitions and enters the shrinking phase.

Exam Theorem: Every schedule standardly adhering to the Two-Phase Locking protocol is guaranteed to be **Conflict Serializable**.

2PL $\Rightarrow$ Conflict Serializable Schedule

Example - Lock Blocking Mechanism:

Transaction 1 (T1)	Transaction 2 (T2)	State & Lock Compatibility Analysis
X(A)	-	T1 acquires Exclusive lock on A.
-	S(A) (Request)	BLOCKED (WAIT): X and S locks are incompatible on item A. T2 waits.
U(A)	-	T1 releases lock on A (Enters Shrinking Phase).
-	S(A) (Granted)	T2 unblocked and acquires Shared lock on A. Ensures order T1 $\rightarrow$ T2 .

5. Comparison: Simple Locking vs. 2PL Protocol

Feature / Dimension	Simple Shared/Exclusive Locking	Two-Phase Locking (2PL)
Shared (S) / Exclusive (X) Locks	Supported	Supported
Explicit Growing Phase	No restriction	Yes (Acquisitions only)
Explicit Shrinking Phase	No restriction	Yes (Releases only)
Post-Unlock Acquisition	Allowed	Strictly Forbidden
Serializability Guarantee	✗ Not Guaranteed	Guaranteed (Conflict Serializable)

6. Exam Traps & Rapid-Fire Q&A

Q1: Can T1 acquire a lock during shrinking phase?

✗ NO. Once shrinking starts, zero acquisitions allowed.

Q2: Can T1 release a lock during growing phase?

✗ NO. Unlocking immediately triggers shrinking phase.

Q3: What event triggers the Shrinking Phase?

The very first unlock (U(X)) operation executed.

Q4: What determines the serial equivalent order?

The ordering of Lock Points among transactions.

Q5: Can T2 get S(A) while T1 holds S(A)?

YES. S-S locks are compatible.

Q6: Can T2 get X(A) while T1 holds S(A)?

✗ NO. S-X locks are incompatible. T2 must wait.

7. One-Minute Quick Revision Summary

 **Memory Trick: "First Take, Then Give"**

8. Drawbacks & Limitations of Basic 2PL

While Basic 2PL guarantees conflict serializability, it **does not solve all concurrency anomalies**. The four major problems are summarized by the acronym **ICDS**.

1. Irrecoverable Schedule ✗

2PL does not force a transaction to hold locks until commit. If **T1** releases a lock, **T2** reads dirty data and commits, but **T1** later aborts, **T2** cannot be rolled back.

2. Cascading Rollback ✗

If multiple transactions (**T2**, **T3**, **T4**) read uncommitted writes of **T1** , an abort by **T1** forces a domino-effect rollback of all dependent transactions.

3. Deadlock ✗

Circular wait conditions can easily occur (e.g., **T1** holds A and waits for B, while **T2** holds B and waits for A). Both transactions freeze indefinitely.

4. Starvation ✗

A transaction requesting an Exclusive lock (**X**) may wait indefinitely if a continuous stream of incoming transactions obtain Shared locks (**S**).

9. Deadlock vs. Starvation Comparison

Dimension	Deadlock	Starvation
Nature of Wait	Infinite / Circular Wait	Very Long / Unfair Wait
Dependency Structure	Circular dependency exists (T1 → T2 → T1)	No circular dependency required
Cause	Transactions waiting on each other	One transaction constantly loses priority to others

10. Summary Table & Exam Checklist

Property / Problem	Basic 2PL Guarantee Status
Conflict Serializability	Guaranteed
Recoverability Guarantee	✗ NOT Guaranteed (Irrecoverable possible)
Cascadeless Schedule Guarantee	✗ NOT Guaranteed (Cascading rollbacks possible)
Freedom from Deadlock	✗ NOT Guaranteed (Deadlocks possible)
Freedom from Starvation	✗ NOT Guaranteed (Starvation possible)

🧠 Golden Memory Trick: I-C-D-S

Basic 2PL solves **Serializability**, but leaves behind **I-C-D-S**:

Irrecoverability | **C**ascading Rollback | **D**eadlock | **S**tarvation.

(Note: Strict 2PL, Rigorous 2PL, and Conservative 2PL are introduced to resolve these specific limitations).